

Lab Practicum 01: Empirical Parallel Computing

Silicon Profiling, Multi-Core Scaling Limits, and Concurrency Hazards

STUDENT NAME Mussa Beknur	STUDENT ID 230103082	HOST CPU / SOC MODEL Intel Core i5-11400H	OPERATING SYSTEM & KERNEL Windows 11 Pro
------------------------------	-------------------------	--	---

Generative AI & Tooling Policy: You are explicitly authorized to use LLMs (ChatGPT, Claude, GitHub Copilot) to generate code scripts in any language (C, C++, Python, Go, Rust, Java). However: All execution timings, memory counters, and hardware cache metrics must be physically benchmarked on your host machine. Fabricated numbers receive zero credit.

TASK 1: Host Silicon Audit & Flynn’s Taxonomy Mapping

15 Minutes | 20 Points

Query your operating system and CPU registers to document physical core topology, cache boundaries, and vector math extensions.

Terminal Audit Commands: Linux: `lscpu && lscpu --caches` | macOS: `sysctl -a | grep machdep.cpu` | Windows: Run `wmic cpu get /format:list` and inspect Task Manager > Performance > CPU.

Hardware Property	Local Measured Value	Hardware Property	Local Measured Value
CPU Model & Architecture	Core i5-11400H	Base & Max Boost Clock	2.70 GHz / 4.50 GHz
Physical Hardware Cores	6	Logical Cores (SMT/Threads)	12
L1 Data / Instruction Cache	L1d: 480KB L1i: 480KB	L2 Cache (Per Core / Total)	1.25 MB / 7.5 MB
L3 Shared Last-Level Cache	12 MB	SIMD Vector Extensions	[SSE4.2, AVX2, AVX-512]

Q1.1: Map your host machine's execution modes into Flynn's Taxonomy:

SISD: Single-threaded sequential code, where a single core executes a single instruction on scalar data in general-purpose registers.

SIMD: Vector computations using AVX-2 and AVX-512 extensions. A single instruction per clock cycle processes a data vector when working with matrices, NumPy, or ML models.

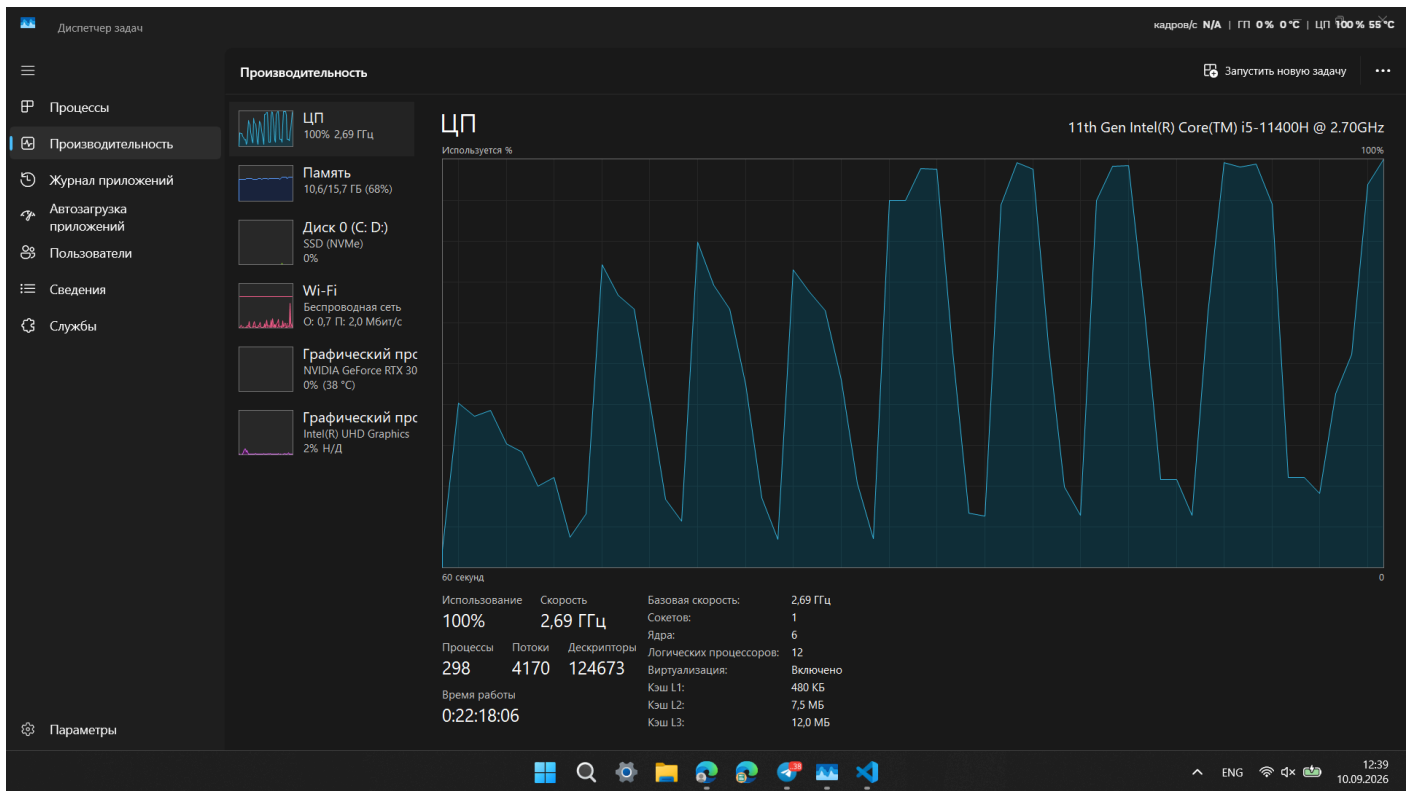
MIMD: Parallel task execution across 6 physical cores / 12 threads. Different cores simultaneously execute independent instruction streams on decomposable datasets.

TASK 2: Multi-Thread Scaling Benchmark & Contention Wall

35 Minutes | 30 Points

Write or prompt an AI to generate a compute-bound CPU benchmark parameterized by thread count N. Run 3 iterations per thread count and log wall-clock execution time.

Workload Algorithm: [e.g., Prime Sieve to 5,000,000 / Matrix Multiply]				Language & Runtime: [e.g., C OpenMP / Python multiprocessing / Go]		
Threads (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Time TN (s)	Speedup SN = T1/TN	Efficiency EN = SN/N
N = 1 (Baseline)	22.78	22.962	22.912	22.884	1.00x	100.0%
N = 2	15.296	15.195	15.054	15.182	1.51x	75.4%
N = 4	9.164	8.862	8.725	8.917	2.57x	64.2%
N = 8	6.315	6.217	6.164	6.232	3.67x	45.9%
N = 16	6.455	6.471	6.568	6.498	3.52x	22.0%
N = 32	7.578	8.590	8.551	8.239	2.78x	8.7%



Q2.1: Scaling Saturation & Hardware Contention Analysis:

Inflection Point: Saturation occurs at N=8 ($S_8 = 3.67x$), with performance degradation reversing at N=16 (6.498s) and N=32 (8.239s, $S_{32} = 2.78x$).

Core/SMT Limits: Peak speedup plateaued around $\sim 3.67x$ due to physical core limits. At $N > 8$, logical SMT threads compete for shared ALU pipelines within the same physical cores, yielding diminishing returns.

OS Overhead: Spawning 16–32 processes exceeds physical pipelines, forcing the OS scheduler into frequent context switching, thread migration, and process management overhead.

Cache Contention: High process counts trigger L3 cache thrashing and memory bus contention, creating a bottleneck that actively slows down execution.

TASK 3: The Unsynchronized Shared Counter & Race Condition Trap

25 Minutes | 25 Points

Spawn 10 parallel threads. Each thread executes a loop incrementing a shared global integer 1,000,000 times without atomic operations or locks. Theoretical expected total: $10 \times 1,000,000 = 10,000,000$. Execute 10 consecutive runs.

Run	Measured Output	Error (10^7 - Act)	Run	Measured Output	Error (10^7 - Act)
#1	2,568,651	7,431,349	#6	3,794,718	6,205,282
#2	3,655,497	6,344,503	#7	4,091,914	5,908,086
#3	4,558,471	5,441,529	#8	3,942,498	6,057,502
#4	3,778,823	6,221,177	#9	3,769,603	6,230,397
#5	4,203,337	5,796,663	#10	3,559,690	6,440,310

Q3.1: Read-Modify-Write (RMW) Window

Incrementing a variable requires three instructions: **Load**, **Add**, and **Store**. Without synchronization, thread operations interleave. When Thread B loads the memory value before Thread A completes its store, both threads write back the same incremented value, causing massive amounts of lost updates.

Q3.2: The Synchronization Penalty

Unlocked = 2465.9 ms vs. Locked = 100231.04 ms
Synchronized code is dramatically slower because locks serialise execution, destroying multi-core parallelism. Additionally, threads incur high overhead from acquire/release locks, context switches while waiting, and cache line invalidation across cores.

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon

15 Minutes | 15 Points

Use your empirical single-thread time (T1) and two-thread time (T2) from Task 2 to derive the parallel fraction p of your workload, then evaluate theoretical performance bounds.

1. Parallel Fraction Derivation (p) Formula: $p = 2 \times (T1 - T2) / T1$ T1 = 22,884s T2 = 15,182s Calculated p = 0.6724	2. Amdahl Theoretical Speedup Ceiling Formula: $S_{max} = 1 / (1 - p)$ Sequential fraction $(1 - p) = 0.3276$ Max Theoretical Speedup $(N \rightarrow \infty) = 3.05\times$
3. Amdahl 64-Core Projection Formula: $S_{64} = 1 / [(1 - p) + (p / 64)]$ Projected Speedup on 64 Cores = 2.96x	4. Gustafson Scaled Speedup (Weak Scaling) Formula: $SGustafson = (1 - p) + (p \times 64)$ Projected Scaled Speedup (64x Problem) = 43.36x

Q4.1: Strong vs. Weak Scaling Analysis:

Amdahl's Law (Strong Scaling) assumes a fixed problem size. As core count increases, the time spent on the parallel fraction decreases to zero, making the fixed sequential fraction ($1 - p = 32.76\%$) a hard bottleneck that caps maximum theoretical speedup at 3.05x, no matter how many cores are added.

Gustafson's Law assumes the problem size grows proportionally with the core count. Since the parallel portion scales up with 64x data while the sequential time stays flat, the sequential overhead becomes negligible relative to the massive parallel work, yielding near-linear scaling.

Why does Gustafson's model predict near-linear scaling on a 64-core node, whereas Amdahl's formula predicts a hard bottleneck for the exact same algorithm?

Submission Guidelines & Deliverables Checklist:

- Complete all 4 tables and technical reflection prompts directly on this document (or submit as an annotated Word / PDF).
- Attach a single terminal screenshot verifying Task 2 executing across active threads (e.g., htop, top, or Task Manager CPU pane).
- Upload the final .docx / PDF to the course portal before the conclusion of the 90-minute lab period.